



WebSocket

SISTEMAS DISTRIBUIDOS

Carlos Rojas Sánchez

Licenciatura en Informática

Universidad del Mar

1. Introducción a WebSocket
2. Cómo Funciona WebSocket
3. WebSocket en Node.js
4. Ejemplo: WebSocket en Node.js

Introducción a WebSocket

WebSocket es un protocolo que permite la comunicación en tiempo real y bidireccional entre el cliente y el servidor a través de una única conexión persistente.

Comparación de Protocolos: HTTP, WebSocket y CORBA

Característica	HTTP	WebSocket	CORBA
Tipo de Comunicación	Unidireccional	Bidireccional	Bidireccional
Conexión	Nueva conexión por solicitud	Conexión persistente	Conexión persistente

Table 1: Comparación entre HTTP, WebSocket y CORBA (Parte 1)

Comparación de Protocolos: HTTP, WebSocket y CORBA

Característica	HTTP	WebSocket	CORBA
Latencia	Mayor	Baja	Baja (pero más pesada que WebSocket)
Protocolo	Basado en texto	Basado en texto/binario	Binario
Escalabilidad	Escalable para contenido estático	Alta escalabilidad en tiempo real	Menos escalable por su complejidad

Table 2: Comparación entre HTTP, WebSocket y CORBA (Parte 2)

Comparación de Protocolos: HTTP, WebSocket y CORBA

Característica	HTTP	WebSocket	CORBA
Facilidad de Implementación	Fácil	Moderada	Compleja (requiere IDL y ORB)
Casos de Uso	Páginas web tradicionales	Chats, juegos en línea	Sistemas distribuidos complejos

Table 3: Comparación entre HTTP, WebSocket y CORBA (Parte 3)

- Chats en tiempo real
- Juegos en línea
- Notificaciones instantáneas
- Aplicaciones financieras (datos en vivo)
- Control remoto de dispositivos IoT

Cómo Funciona WebSocket

Proceso de conexión (Handshake)

1. El cliente inicia una conexión HTTP estándar.
2. El cliente envía una cabecera Upgrade: websocket para solicitar el cambio de protocolo.
3. El servidor responde con el código 101 Switching Protocols y se establece la conexión WebSocket.

- Mensajes en formato texto o binario.
- Ambos extremos pueden enviar datos de forma asíncrona.

WebSocket en Node.js

1. Inicializa tu proyecto
2. Instala la librería ws

- .on()** Escuchar eventos. Se ejecuta cada vez que se recibe el evento especificado.
- .emit()** Enviar eventos. Se usa para enviar eventos junto con datos opcionales.

[Flujo de datos]

1. Cliente envía mensaje → `socket.emit()`
2. Servidor recibe → `socket.on()`
3. Servidor reenvía a otros clientes → `io.emit()`
4. Cliente recibe el mensaje de vuelta → `socket.on()`

WebSocket Protocolo nativo.

Socket.IO Una implementación más avanzada de WebSocket.

Eventos en WebSocket

Evento	Descripción
open	Se activa cuando la conexión se establece.
message	Se activa cuando se recibe un mensaje del otro extremo.
error	Se activa cuando ocurre un error en la conexión.
close	Se activa cuando la conexión se cierra.

Evento	Descripción
<code>connect</code>	Se activa cuando un cliente se conecta correctamente.
<code>disconnect</code>	Se activa cuando un cliente se desconecta del servidor.
<code>disconnecting</code>	Se activa justo antes de que un cliente se desconecte.
<code>error</code>	Se activa cuando ocurre un error en la conexión.

Evento	Descripción
<code>connect_error</code>	Se dispara si ocurre un error durante el intento de conexión.
<code>connect_timeout</code>	Se dispara si la conexión excede el tiempo de espera configurado.
<code>reconnect</code>	Se activa cuando el cliente se reconecta después de una desconexión.
<code>reconnect_attempt</code>	Se dispara cuando el cliente intenta reconectarse.

Eventos en Socket.IO

Evento	Descripción
<code>reconnect_error</code>	Se dispara si ocurre un error durante el intento de reconexión.
<code>reconnect_failed</code>	Se dispara si el cliente no logra reconectarse después de varios intentos.
<code>ping</code>	Se envía automáticamente para comprobar la latencia de la conexión.
<code>pong</code>	Responde automáticamente a un <code>ping</code> .
Eventos personalizados	Definidos por el desarrollador para manejar lógica específica.

Ejemplo: WebSocket en Node.js

Estructura del Proyecto

```
websocket-ejemplo/  
|  
├── server.js  
├── package.json  
├── public/  
├── index.html  
├── style.css  
└── app.js
```

Crear proyecto

```
mkdir websocket-ejemplo  
cd websocket-ejemplo  
npm init -y
```

Instalar WebSocket

```
npm install ws
```

Servidor Node.js

Archivo: `server.js`

```
const http = require("http");  
const fs = require("fs");  
const path = require("path");  
const WebSocket = require("ws");
```

- Se crea un servidor HTTP.
- Se importa la librería WebSocket.

Servidor HTTP

```
const server = http.createServer((req, res) => {  
  
  let filePath = "./public/index.html";  
  
  if (req.url !== "/") {  
    filePath = "./public" + req.url;  
  }  
  
});
```

- El servidor entrega archivos HTML, CSS y JS.

Crear Servidor WebSocket

```
const wss = new WebSocket.Server({ server });  
  
wss.on("connection", (ws) => {  
  
    console.log("Cliente conectado");  
  
});
```

- Se detectan clientes conectados.

```
ws.on("message", (message) => {  
    console.log(message.toString());  
});
```

- El servidor recibe mensajes enviados por clientes.

Enviar Mensajes a Todos

```
wss.clients.forEach((client) => {  
    if(client.readyState === WebSocket.OPEN) {  
        client.send(message.toString());  
    }  
});
```

- El mensaje se reenvía a todos los clientes.

Archivo: index.html

```
<div class="contenedor">
<h1>Chat WebSocket</h1>
<div id="mensajes"></div>
<input type="text" id="texto">
<button id="btnEnviar">
Enviar
</button>
</div>
```

Archivo: `style.css`

```
body {  
    font-family: Arial;  
    background: #f0f2f5;  
}
```

```
.contenedor {  
    width: 400px;  
    background: white;  
}
```

- El CSS define la apariencia visual.

Archivo: app.js

```
const socket =  
new WebSocket("ws://localhost:3000");
```

- El navegador se conecta al servidor.

Recibir Mensajes en Cliente

```
socket.onmessage = (event) => {  
  const div =  
    document.createElement("div");  
  div.textContent = event.data;  
  mensajes.appendChild(div);  
};
```

- Los mensajes aparecen dinámicamente en pantalla.


```
btnEnviar.addEventListener("click", () => {  
    socket.send(texto.value);  
});
```

- El cliente envía texto al servidor.

Ejecutar Proyecto

Iniciar servidor

```
node server.js
```

Abrir navegador

```
http://localhost:3000
```

- Abrir varias pestañas para probar el chat.

- Comunicación en tiempo real.
- Baja latencia.
- Conexión persistente.
- Ideal para chats y juegos.